

Week 2: Branching Workflow, Google Auth, and the Onboarding Flow

Yash

Roll No: 1024030846

4 September 2026

1 Context

Following on from Week 1's scaffolding and CI/CD pipeline, this week's work covered three things: putting a real branching discipline in place, replacing the placeholder authentication approach with the one actually needed for production, and designing *and* implementing the first real feature – new-user onboarding – using a structured, review-gated development process.

2 Branching Workflow

Set up a proper `Dev` → `feature/*` branching model on GitHub, since the team will now be working in parallel:

- `main` – protected, production, requires a PR with at least 1 approval.
- `Dev` – protected integration branch, requires a PR (self-merge allowed for speed), deploys as a Vercel *preview*, not production.
- `feature/<name>` – cut from `Dev`, one per task, PR'd back into `Dev`.

Also updated the CI/CD workflow so pushes to `Dev` deploy as preview rather than accidentally matching the old production-deploy condition, which only checked "is this a pull request" rather than "is this specifically `main`".

3 Database: Prisma Schema for Onboarding

Added five new fields to the `Profile` model and two new enums (`ComfortLevel`, `Availability`), migrated against the real Neon Postgres database set up in Week 1. This schema supports both onboarding paths described below without needing a second migration later.

4 Authentication: From GitHub-as-Login to College Google Sign-In

The original plan (and Week 1's implementation) used GitHub OAuth as the sign-in method. On review, this was the wrong design: the platform's actual requirement is that only verified college students can sign up, and GitHub accounts carry no such guarantee.

Redesign: sign-in is now Google-only, using `Auth.js v5`, restricted to a configurable college email domain via a `signIn` callback:

```

callbacks: {
  async signIn({ user }) {
    if (!ALLOWED_EMAIL_DOMAIN) return true;
    return Boolean(
      user.email?.endsWith(`@${ALLOWED_EMAIL_DOMAIN}`)
    );
  },
}

```

GitHub is no longer part of login at all – it is deferred to a later, optional “connect your GitHub” step, used purely as an evidence source once a student is already signed in.

4.1 Version choice: Auth.js v5 over the “latest” tag

`npm install next-auth` resolves `latest` to a 4.x release built for the older Pages Router; the App Router integration works only through a compatibility shim. Pinned instead to `next-auth@5.0.0-beta.32` (Auth.js v5) – despite the “beta” label, this is the version actually designed for the App Router and is what essentially every current Next.js tutorial and real project uses.

5 Feature: New-User Onboarding Flow

Designed and implemented the first real product feature: after a student’s first Google sign-in, they are routed to a single `/onboarding` screen with two paths.

5.1 Design

- **Path A** (has a resume/GitHub): uploads a resume, stored in Vercel Blob, creating an `EvidenceRecord` with `status: "pending-parse"` for a future parsing pipeline. GitHub connection is a visible but disabled “coming soon” button for now.
- **Path B** (Explorer/manual): self-reports comfort level, interest tags, availability, and optional project links, written directly onto `Profile`.

A key design principle from the project spec: self-reported data (Path B) is *not* treated as evidence – only Path A creates an `EvidenceRecord`. This was written into the design spec before implementation and verified afterwards against the real database.

Route gating was deliberately implemented as a server component layout (`app/(protected)/layout.tsx`) rather than Next.js middleware, because middleware runs on the Edge runtime by default, and Prisma’s client needs the Node runtime – using middleware would have required a second, HTTP-based database driver just for this one check.

5.2 Process: Design → Plan → Subagent-Driven Implementation

Rather than writing the feature directly, followed a structured process: a written design spec, a detailed implementation plan broken into 7 tasks with exact interfaces between them, then execution via a fresh, isolated AI subagent per task, with an independent review (spec compliance *and* code quality) after every task, plus one final whole-branch review at the end. Every task passed its individual review clean or after a small fix round.

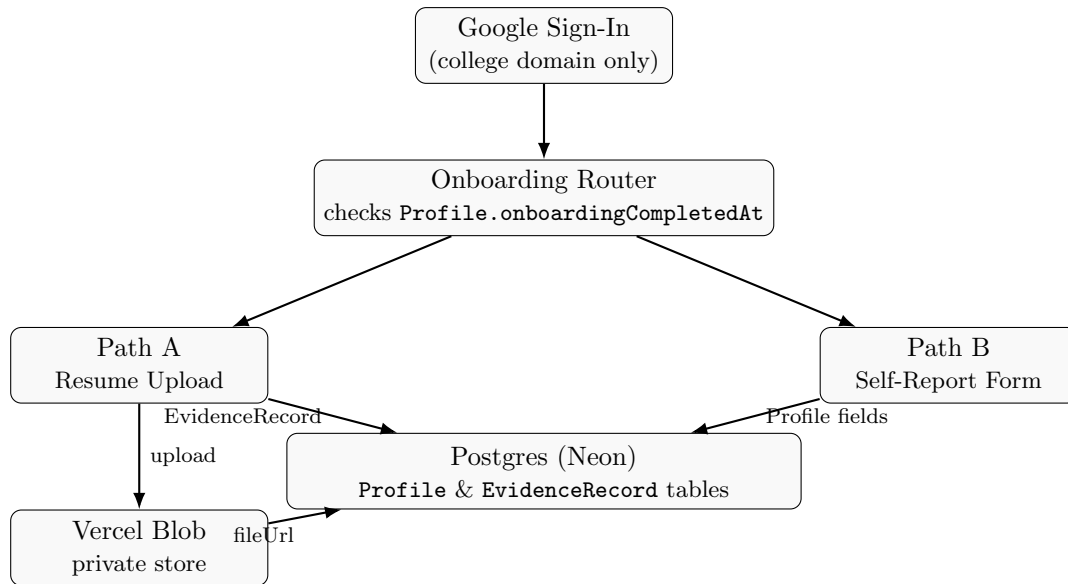


Figure 1: Onboarding data flow. Path A writes to both Vercel Blob and Postgres inside one transaction; Path B writes only to Postgres and never touches Blob storage.

5.3 Debugging: Public vs. Private Blob Storage

Error encountered: an automated security scan flagged that resumes (personal, identifying documents) were being stored with `access: "public"` on Vercel Blob – readable by anyone with the URL, forever.

Relevant context: the original plan specified `access: "public"` based on an outdated assumption that the Vercel Blob SDK only supported public objects.

Key observation: checking the actually-installed SDK version’s type definitions directly (`node_modules/@vercel/blob/dist/index.d.ts`) showed full support for `access: "private"`, where the object can only be read back server-side using the project’s own `BLOB_READ_WRITE_TOKEN` – not by anyone with the URL. This also explained why an earlier upload attempt had failed with `Cannot use public access on a private store`: the actual Blob store was already configured as private, and the code was wrong, not the store.

Solution: switched `uploadResume` to `access: "private"`. No dashboard changes, no rework – purely a code correction, and strictly more secure than the original plan.

5.4 Debugging: The Vanishing 5MB Resume Limit

This was the most valuable finding of the week, and it only surfaced because of the *final*, whole-branch review – no single task’s isolated review could have seen it, since it lives in the gap between three different tasks’ code.

Error (as it would have shown up in production): a user uploading a genuinely resume-sized PDF (over 1MB, common once a resume has any formatting or a headshot) would have the upload silently rejected by the framework, despite the UI advertising “max 5MB” and the server code correctly checking for exactly that limit.

Relevant context: Next.js caps the request body of a Server Action at 1MB *by default*, independently of any application-level size check:

```
const nextConfig: NextConfig = {
  /* config options here -- untouched default */
};
```

Key observation: the live, manual walkthrough of the feature used a small test PDF and passed completely – the bug was invisible to functional testing precisely because the test file happened to be under 1MB. It took a reviewer reading the Next.js documentation shipped inside `node_modules` itself to notice the framework-level cap existed at all.

Solution: raised the limit explicitly in `next.config.ts`:

```
experimental: {
  serverActions: {
    bodySizeLimit: "5mb",
  },
},
```

Also added a matching client-side size/type check (so a bad file fails fast, before upload) and wrapped the resume-upload database transaction in a `try/catch` so an infrastructure failure (e.g. a Blob outage) returns a clean inline error instead of crashing the page – being careful that the `catch` block does *not* swallow Next.js’s internal redirect mechanism, which itself works by throwing a special error.

Because per-task code review only ever sees one task’s diff at a time, a bug that spans the interaction of three separate files (UI copy, server validation, and a build config file none of the three tasks individually touched) can only be caught by a review that looks at the whole branch at once. This is the concrete justification for running that final broad review rather than treating “all tasks passed” as equivalent to “the feature is correct.”

6 Verification

No automated test framework exists in this codebase yet, so verification was manual throughout: TypeScript compiles cleanly, the production build succeeds, and – most importantly – a live walkthrough against the real Neon database and real Google accounts confirmed:

- Path A (real PDF upload) correctly creates an `EvidenceRecord` and marks onboarding complete.
- Path B correctly populates the self-reported fields and creates *no* evidence record.
- Revisiting `/onboarding` after completion redirects away.
- A signed-out visitor still sees the ordinary sign-in page, unaffected by the new gating logic.

7 Outcome

Opened a pull request into `Dev` containing the full onboarding feature, the design spec, and the implementation plan as project documentation.

8 Follow-ups

- Interest tags are currently stored as their display labels rather than stable ids – fine at zero data, worth normalising before real usage accumulates.
- Resume parsing (turning an uploaded PDF into structured skill evidence) is still unbuilt; uploads are stored with a `"pending-parse"` status for now.
- Real GitHub-account linking (as an evidence source, not login) remains a separate, not-yet-started piece of work.